

A **software bug** is a defect (bug) in computer software. A computer program with many or serious bugs may be described as *buggy*.

The effects of a software bug range from minor (such as a misspelled word in the user interface) to severe (such as frequent crashing).

In 2002, a study commissioned by the US Department of Commerce's National Institute of Standards and Technology concluded that "software bugs, or errors, are so prevalent and so detrimental that they cost the US economy an estimated \$59 billion annually, or about 0.6 percent of the gross domestic product".^[1]

Since the 1950s, some computer systems have been designed to detect or auto-correct various software errors during operations.

History

The term *bug* to describe a defect has been engineering jargon since at least as far back as the 1870s, long before electronic computers and computer software. For instance, Thomas Edison wrote the following words in a letter to an associate in 1878:

	It has been just so in all of my inventions. The first step is an intuition, and comes with a burst, then difficulties arise—this thing gives out and [it is] then that "Bugs"—as such little faults and difficulties are called—show themselves and months of intense watching, study and labor are requisite before commercial success or failure is certainly reached.^[2]
	

Problems with military gear during World War II were referred to as bugs or glitches.^[3]

Isaac Asimov used the term *bug* to relate to issues with a robot in his short story "Catch That Rabbit," published in 1944.

U.S. Navy Rear Adm. Grace Hopper, a computer pioneer, popularized a story about a moth that caused a problem in an early electromechanical computer.^[4] While Hopper was working on the Mark II and Mark III as Harvard faculty in about 1947, operators traced an error in the Mark II to a moth trapped in a relay. The moth was removed from the mechanism and taped in a log book with the note "First actual case of bug being found." ^[5] Reportedly, the operators, including William "Bill" Burke, later of the Naval Weapons Laboratory, Dahlgren, Virginia,^[6] were familiar with the engineering term and probably making a pun by conflating the two meanings of *bug* (biological and technical). Even if a joke, the story indicates that the term was commonly used in the computer field at that time.^{[7][8][9][10]}

Terminology

Mistake metamorphism (from Greek *meta* = "change", *morph* = "form") refers to the evolution of a defect in the final stage of software deployment. Transformation of a *mistake* committed by an analyst in the early stages of the software development lifecycle, which leads to a *defect* in the final stage of the cycle has been called *mistake metamorphism*.^[11]

Different stages of a mistake in the development cycle may be described as mistake,^{[12]:31} anomaly,^{[12]:10} fault,^{[12]:31} failure,^{[12]:31} error,^{[12]:31} exception,^{[12]:31} crash,^{[12]:22} glitch, bug,^{[12]:14} defect, incident,^{[12]:39} or side effect.

Examples

Software bugs have been linked to disasters.

- Software bugs in the Therac-25 radiation therapy machine were directly responsible for patient deaths in the 1980s.^[13]
- In 1996, the European Space Agency's US\$1 billion prototype Ariane 5 rocket was destroyed less than a minute after launch due to a bug in the on-board guidance computer program.^[14]
- In 1994, an RAF Chinook helicopter crashed, killing 29; this was initially blamed on pilot error, but was later thought to have been caused by a software bug in the engine-control computer.^[15]
- Buggy software caused the early 21st century British Post Office scandal.^[16]

Controversy

Sometimes the use of *bug* to describe the behavior of software is contentious due to perception. Some suggest that the term should be abandoned, contending that *bug* implies that the defect arose on its own, and push to use *defect* instead, since it more clearly indicates they are caused by a human.^[17]

Some contend that *bug* may be used to cover up an intentional design decision. In 2011, after receiving scrutiny from US Senator Al Franken for recording and storing users' locations in unencrypted files,^[18] Apple called the behavior a bug. However, Justin Brookman of the Center for Democracy and Technology directly challenged that portrayal, stating, "I'm glad that they are fixing what they call bugs, but I take exception with their strong denial that they track users."^[19]

Prevention

Preventing bugs as early as possible in the software development process is a target of investment and innovation.^{[20][21]}

Language support

Newer programming languages tend to be designed to prevent common bugs based on vulnerabilities of existing languages. Lessons learned from older languages such as BASIC and C are used to inform the design of later languages such as C# and Rust.

A compiled language allows for detecting some typos (such as a misspelled identifier) before runtime which is earlier in the software development process than for an interpreted language.

Languages may include features such as a static type system, restricted namespaces, and modular programming. For example, for a typed, compiled language (like C):

```
float num = "3";
```

is syntactically correct, but fails type checking since the right side, a string, cannot be assigned to a float variable. Compilation fails – forcing this defect to be fixed before development progress can resume. With an interpreted language, a failure would not occur until the statement was executed (much later, if ever, during runtime).

Some languages exclude features that easily lead to bugs, at the expense of slower performance – the principle being that it is usually better to write simpler, slower correct code than complicated, buggy code. For example, Java does not support pointer arithmetic which can be very fast but may lead to memory corruption or segmentation faults if not used with great caution.

Some languages include features that add runtime overhead in order to prevent common bugs. For example, many languages include runtime bounds checking and a way to recover from out-of-bounds errors instead of crashing.

Techniques

Style guidelines and defensive programming can prevent easy-to-miss typographical errors (typos).

For example, most C-family programming languages allow the omission of braces around an instruction block if there's only a single instruction. The following code executes function foo only if condition is true:

```
if (condition)
    foo();
```

But this code always executes foo:

```
if (condition);
foo();
```

Using braces – even if they're not strictly required – reliably prevents this error:

```
if (condition) {
    foo();
}
```

Enforcement of conventions may be manual (i.e. via code review) or via automated tools such as linters.

Specification

Some contend that writing a program specification, which states the intended behavior of a program, can prevent bugs. Others, however, contend that formal specifications are impractical for anything but the shortest programs, because of problems of combinatorial explosion and indeterminacy.

Software testing

One goal of software testing is to find bugs. Measurements during testing can provide an estimate of the number of likely bugs remaining. This becomes more reliable the longer a product is tested and developed.^[22]

Agile practices

Agile software development may involve frequent software releases with relatively small changes. Defects are revealed by user feedback.

With test-driven development (TDD), unit tests are written while writing the production code, and the production code is not considered complete until all tests have been written and complete successfully.

Static analysis

Tools for static code analysis help developers by inspecting the program text beyond the compiler's capabilities to spot potential problems. Although in general the problem of finding all programming errors given a specification is not solvable (see halting problem), these tools exploit the fact that human programmers tend to make certain kinds of simple mistakes when writing software.

Instrumentation

Tools to monitor the performance of the software as it is running, either specifically to find problems such as bottlenecks or to give assurance as to correct working, may be embedded in the code explicitly (perhaps as simple as a statement saying PRINT "I AM HERE"), or provided as tools. It is often a surprise to find that most of the time is taken by a piece of code, and this removal of assumptions might cause the code to be rewritten.

Open source

Open source development allows anyone to examine source code. A school of thought popularized by Eric S. Raymond as Linus's law says that popular open-source software has more chance of having few or no bugs than other software, because "given enough eyeballs, all bugs are shallow".^[23] This assertion has been disputed, however: computer security specialist Elias Levy wrote that "it is easy to hide vulnerabilities in complex, little understood, and undocumented source code," because, "even if people are reviewing the code, that doesn't mean they're qualified to do so."^[24] An example of an open-source software bug was the 2008 OpenSSL vulnerability in Debian.

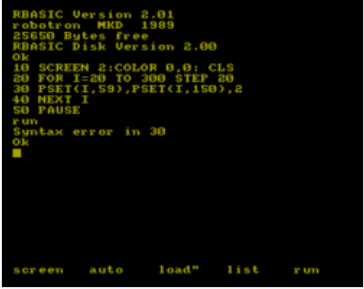
Debugging

Debugging can be a significant part of the software development lifecycle. Maurice Wilkes, an early computing pioneer, described his realization in the late 1940s that "a good part of the remainder of my life was going to be spent in finding errors in my own programs" .^[25]

Typically, the first step in locating a bug is to reproduce it reliably. If unable to reproduce the issue, a programmer cannot find the cause of the bug and therefore cannot fix it.

Some bugs are revealed by inputs that may be difficult to reproduce. One cause of the Therac-25 radiation machine deaths was a bug (specifically, a race condition) that occurred only when the machine operator very rapidly entered a treatment plan; it took days of practice to become able to do this, so the bug did not manifest in testing or when the manufacturer attempted to reproduce it. Other bugs may stop occurring whenever the setup is augmented to help find the bug, such as running the program with a debugger; these are called *heisenbugs* (humorously named after the Heisenberg uncertainty principle).

Sometimes, a bug is not an isolated flaw, but represents an error of thinking or planning on the part of the programmers. Often, such a *logic error* requires a section of the program to be overhauled or rewritten.



A BASIC program failing due to a syntax error in its code



A page from the Harvard Mark II electromechanical computer's log, featuring a dead moth that was removed from the device



Error resulting from a software bug displayed on two screens at La Croix de Berry station in France

A **code review**, stepping through the code, and imagining or transcribing the execution process, may often find errors without ever reproducing the bug as such.

A program known as a **debugger** can help a programmer find faulty code by examining the inner workings of a program, such as executing code line-by-line and viewing variable values.

As an alternative to using a debugger, code may be instrumented with logic to output debug information to trace program execution and view values. Output is typically to **console**, **window**, **log file**, or a **hardware** output, potentially driving an indicator **LED**.

Since the 1990s, particularly following the **Ariane 5 Flight 501** disaster, interest in automated aids to debugging rose, such as **static code analysis** by **abstract interpretation**.^[26]

In an **embedded system**, the software is often modified to **work around** a hardware bug since software modifications can be cheaper and less disruptive than modifying the hardware.

Management

Bugs are managed via activities like documenting, categorizing, assigning, reproducing, correcting, and releasing the corrected code.

Tools are often used to track bugs and other issues with software. Typically, different tools are used by the software development team to **track their workload** than by **customer service** to **track user feedback**.^[27]

A tracked item is often called *bug*, *defect*, *ticket*, *issue*, *feature*, or for agile software development, *story*, or *epic*. Items are often categorized by aspects such as severity, priority, and version number(s) affected.

In a process sometimes called **triage**, choices are made for each bug about whether and when to fix it based on information such as the bug's severity and priority and external factors such as development schedules. Triage generally does not include an investigation into the cause. Triage may occur regularly and minimally consists of reviewing new bugs since the previous triage, possibly extending to all open bugs. Attendees may include the project manager, the development manager, the test manager, the build manager, and technical experts.^{[28][29]}

Severity

Severity is a measure of the impact the bug has.^[30] This impact may be data loss, financial, loss of goodwill, and wasted effort. Severity levels are not standardized, but differ by context, such as industry and tracking tool. For example, a crash in a video game has a different impact than a crash in a bank server. Example severity level descriptions are *crash or hang*, *no workaround* (user cannot accomplish a task), *has workaround* (user can still accomplish the task), *visual defect* (a misspelling for example), or *documentation error*. Another example set of severities: *critical*, *high*, *low*, *blocker*, *trivial*.^[31] The severity of a bug may be the same category as its priority for being fixed, or the two may be quantified and managed separately.

A bug severe enough to delay the release of the product is called a *show stopper*.^{[32][33]}

Priority

Priority describes the importance of resolving the bug in relation to other bugs. Priorities might be numerical, such as 1 through 5, or named, such as *critical*, *high*, *low*, and *deferred*. The values might be similar or identical to severity ratings, even though priority is a different aspect.

Priority may be a combination of the bug's severity and the level of effort to fix. A bug with low severity but easy to fix may get a higher priority than a bug with moderate severity that requires significantly more effort to fix.

Patch

A bug of sufficiently high priority may warrant a special release to fix it. Such a release is sometimes called a *patch*.

Maintenance release

A software release that emphasizes bug fixes may be called a *maintenance* release – to differentiate it from a release that emphasizes new features or other changes.

Known issue

It is common practice to release software with known, low-priority bugs or other issues. Possible reasons include but are not limited to:

- A deadline must be met and resources are insufficient to fix all bugs by the deadline^[34]
- The bug is already fixed in an upcoming release, and it is not of high priority
- It may be suspected, or known, that some users are relying on the existing buggy behavior; a proposed fix may introduce a **breaking change**
- The problem is in an area that will be obsolete with an upcoming release; fixing it is unnecessary
- "It's not a bug, it's a feature"^[35] A misunderstanding exists between expected and actual behavior or **undocumented features**.

Implications

The amount and type of damage a software bug may cause affects decision-making, processes, and policy regarding software quality. In applications such as human spaceflight, aviation, nuclear power, health care, public transport, or automotive safety, since software flaws have the potential to cause human injury or even death, such software will have far more scrutiny and quality control than, for example, an online shopping website. In applications such as banking, where software flaws have the potential to cause serious financial damage to a bank or its customers, quality control is also more important than, say, a photo editing application.

Other than the damage caused by bugs, some of their cost is due to the effort invested in fixing them. In 1978, Lientz et al. showed that the median of projects invested 17 percent of their development effort in bug fixing.^[36] In 2020, research on **GitHub** repositories showed the median is 20 percent.^[37]

Metrics

In 1994, NASA's **Goddard Space Flight Center** managed to reduce their average number of errors from 4.5 per 1,000 lines of code (**SLOC**) to 1 per 1000 SLOC.^[38]

Another study in 1990 reported that exceptionally good software development processes can achieve deployment failure rates as low as 0.1 per 1000 SLOC.^[39] This figure is iterated in literature such as *Code Complete* by Steve McConnell,^[40] and the *NASA study on Flight Software Complexity*.^[41] Some projects even attained zero defects: the **firmware** in the IBM Wheelwriter typewriter which consists of 63,000 SLOC, and the **Space Shuttle** software with 500,000 SLOC.^[39]

To facilitate reproducible research on testing and debugging, researchers use curated benchmarks of bugs:

- the Siemens benchmark^[42]
- ManyBugs is a benchmark of 185 C bugs in nine open-source programs.^[43]
- Defects4J is a benchmark of 341 Java bugs from 5 open-source projects.^[44] It contains the corresponding patches, which cover a variety of patch types. It has been largely used for research on automatic program repair.^[45]

Types

Some notable types of bugs:

Design error

A bug can be caused by insufficient or incorrect design based on the specification. For example, given that the specification is to alphabetize a list of words, a design bug might occur if the design does not account for symbols, resulting in incorrect alphabetization of words with symbols.

Arithmetic

Numerical operations can result in unexpected output, slow processing, or crashing.^[46] Such a bug can be from a lack of awareness of the qualities of the data storage such as a **loss of precision** due to **rounding**, **numerically unstable** algorithms, **arithmetic overflow** and **underflow**, or from lack of awareness of how calculations are handled by different software coding languages such as **division by zero** which in some languages may **throw an exception**, and in others may return a special value such as **NaN** or **infinity**.

Control flow

A **control flow** bug, or **logic error**, is characterized by code that does not fail with an error, but does not have the expected behavior, such as **infinite looping**, **infinite recursion**, **incorrect comparison** in a **conditional** such as using the wrong **comparison operator**, and the **off-by-one error**.

Interfacing

- Incorrect API usage.
- Incorrect protocol implementation.
- Incorrect hardware handling.
- Incorrect assumptions of a particular platform.
- Incompatible systems. A new API or communications protocol may seem to work when two systems use different versions, but errors may occur when a function or feature implemented in one version is changed or missing in another. In production systems which must run continually, shutting down the entire system for a major update may not be possible, such as in the telecommunication industry^[47] or the internet.^{[48][49][50]} In this case, smaller segments of a large system are upgraded individually, to minimize disruption to a large network. However, some sections could be overlooked and not upgraded, and cause compatibility errors that may be difficult to find and repair.
- Incorrect code annotations.

Concurrency

- Deadlock** – a task cannot continue until a second finishes, but at the same time, the second cannot continue until the first finishes.
- Race condition** – multiple simultaneous tasks compete for resources.
- Errors in **critical sections**, **mutual exclusions**, and other features of **concurrent processing**. **Time-of-check to time-of-use** (TOCTOU) is a form of unprotected critical section.

Resourcing

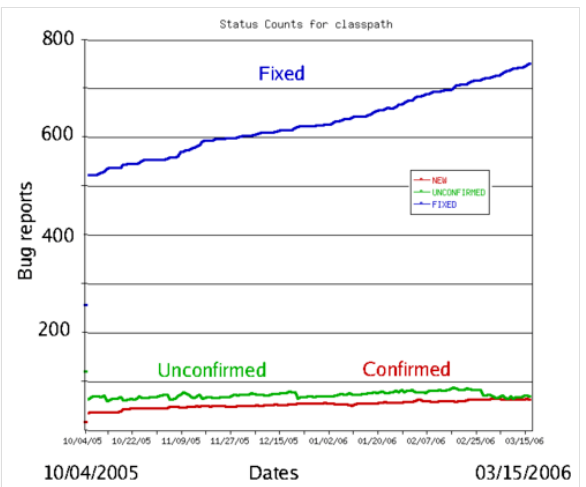
- Null **pointer** dereference.
- Using an **uninitialized variable**.
- Using an otherwise valid instruction on the wrong **data type** (see **packed decimal**/**binary-coded decimal**).
- Access violations**.
- Resource leaks**, where a finite system resource (such as **memory** or **file handles**) becomes exhausted by repeated allocation without **release**.
- Buffer overflow**, in which a program tries to store data past the end of allocated storage. This may or may not lead to an access violation or **storage violation**. These are frequently **security bugs**.
- Excessive recursion which—though logically valid—causes **stack overflow**.
- Use-after-free error**, where a **pointer** is used after the system has freed the memory it references.
- Double free error.

Syntax

- Use of the wrong **token**, such as performing assignment instead of **equality test**. For example, in some languages x=5 will set the value of x to 5 while x==5 will check whether x is currently 5 or some other number. Interpreted languages allow such code to fail. Compiled languages can catch such errors before testing begins.

Teamwork

- Unpropagated updates: for example, when a programmer changes "myAdd" but forgets to change "mySubtract", which uses the same algorithm. These errors are mitigated by the **Don't Repeat Yourself** philosophy.
- Comments out of date or incorrect: many programmers assume the comments accurately describe the code.
- Differences between documentation and product.


 Example bug history (GNU Classpath project data). A new bug is initially *unconfirmed*. Once reproduced, it is changed to *confirmed*. Once the issue is resolved, it is changed to *fixed*.

